

NiceGUI app.storage.client 全面详细解析

`app.storage.client` 是 NiceGUI 内置的五种核心存储类型之一，专为**客户端会话级临时存储**设计，核心特点是“服务器端内存存储、客户端专属隔离、随页面访问周期销毁”，适用于存储仅需在当前页面访问期间保留的临时数据（如短期缓存、页面级状态、资源密集型对象），无需手动清理，兼顾数据隔离与资源效率。

一、核心定位与设计目标

1. 核心价值

- 客户端专属隔离**：每个客户端（浏览器 / 设备连接）拥有独立的 `app.storage.client` 存储实例，数据仅当前客户端可见，不与其他客户端或标签页共享；
- 服务器端灵活存储**：数据存储在服务器内存中，支持存储任意 Python 对象（如数据库连接、流对象、自定义类实例），无浏览器存储的序列化限制和容量约束；
- 自动生命周期管理**：数据随客户端当前页面访问周期终结而销毁（页面重载、导航离开、浏览器关闭时自动清理），无需手动释放资源，避免内存泄漏；
- 轻量临时缓存**：适合存储短期使用、无需持久化的资源（如页面加载时的临时数据、动态生成的缓存结果），平衡性能与资源占用。

2. 设计目标

- 解决“客户端短期临时数据存储”需求，避免将临时数据混入长期存储（如 `app.storage.user`）或全局存储（如 `app.storage.general`）；
- 支持资源密集型对象存储（如数据库连接、WebSocket 连接），且无需担心跨客户端共享导致的并发问题；
- 简化临时数据管理：客户端页面访问结束后自动清理数据，减少手动清理代码；
- 适配“页面刷新后恢复默认状态”的场景（如每次重载页面都使用初始配置，而非保留上次状态）。

二、核心特性与关键规则

1. 存储基础信息

特性	详细说明
存储位置	服务器端内存（非磁盘文件、非浏览器存储），数据仅驻留内存，不持久化到磁盘
数据支持	可存储任意 Python 对象（基础类型、自定义类实例、数据库连接、流对象、缓存结果等），无需序列化 / 反序列化
生命周期	客户端当前页面访问周期：从客户端连接建立、页面加载开始，到页面重载、导航至其他页面、浏览器关闭或客户端断开连接时终结
共享范围	仅当前客户端的当前页面访问有效，不跨客户端、不跨标签页、不跨页面重载、不跨浏览器
依赖条件	无需等待客户端连接就绪（区别于 <code>app.storage.tab</code> ），页面加载后即可直接操作

特性	详细说明
自动清理	客户端页面访问周期结束后，框架自动销毁存储实例及其中数据，释放服务器内存

2. 关键使用规则

- 生命周期边界：**核心区别于其他存储类型的关键 —— 页面重载、导航到应用内其他页面、关闭浏览器标签页，都会导致 `app.storage.client` 数据被清空；仅在当前页面的“无刷新交互”（如按钮点击、组件联动）中保持数据；
- 无持久化特性：**服务器重启、客户端断开连接后，数据直接丢失，不支持跨会话、跨重启保留；
- 客户端隔离粒度：**同一浏览器的不同标签页访问同一应用，视为两个独立客户端，各自的 `app.storage.client` 数据完全隔离；
- 无需序列化：**因存储在服务器内存，无需像 `app.storage.browser` 或 `app.storage.user` 那样要求数据可序列化，支持复杂对象直接存储。

三、核心 API 与基础用法

1. 基础操作 API

`app.storage.client` 的 API 完全模仿 Python 字典（`dict`），支持键值对的增、删、改、查，操作简洁直观，无需额外配置：

操作	语法	功能描述	示例
读取值	<code>app.storage.client[key]</code> 或 <code>app.storage.client.get(key, default)</code>	根据键读取值， <code>get</code> 方法支持默认值（键不存在时返回）	<code>count = app.storage.client.get('count', 0)</code>
写入 / 更新值	<code>app.storage.client[key] = value</code>	写入新键值对或更新已有键的值（内存操作，无磁盘 I/O）	<code>app.storage.client['temp_data'] = {'name': 'test'}</code>
删除值	<code>del app.storage.client[key]</code> 或 <code>app.storage.client.pop(key, default)</code>	删除指定键， <code>pop</code> 方法支持默认值（键不存在时返回）	<code>app.storage.client.pop('cache_result', None)</code>
检查键存在	<code>key in app.storage.client</code>	判断键是否存在于当前客户端存储中	<code>if 'db_conn' in app.storage.client: ...</code>
清空存储	<code>app.storage.client.clear()</code>	手动清空当前客户端的所有存储数据	<code>app.storage.client.clear()</code>
获取长度	<code>len(app.storage.client)</code>	返回当前客户端存储的键值对数量	<code>data_count = len(app.storage.client)</code>
批量更新	<code>app.storage.client.update(**kwargs)</code>	批量写入多个键值对	<code>app.storage.client.update(a=1, b=2)</code>

2. 基础使用示例（无依赖快速上手）

由于 `app.storage.client` 无需等待客户端连接就绪，可直接在页面函数中操作，上手成本极低：

```
from nicegui import app, ui

@ui.page('/')
def index():
    # 初始化客户端存储的计数器（键不存在时设为 0）
    app.storage.client['count'] = app.storage.client.get('count', 0)

    # 绑定标签展示计数器值
    count_label = ui.label(f'当前点击次数: {app.storage.client["count"]}')

    # 按钮点击时更新计数器（仅当前客户端有效）
    def increment_count():
        app.storage.client['count'] += 1
        count_label.set_text(f'当前点击次数: {app.storage.client["count"]}')
    ui.button('点击递增', on_click=increment_count)

    # 重置按钮（清空当前客户端的计数器）
    def reset_count():
        app.storage.client['count'] = 0
        count_label.set_text(f'当前点击次数: {app.storage.client["count"]}')
    ui.button('重置', on_click=reset_count)

    # 重载页面按钮（验证数据是否丢失）
    ui.button('重载页面', on_click=ui.navigate.reload)

ui.run()
```

效果说明：点击“点击递增”按钮，计数器累加；同一浏览器打开新标签页（新客户端），计数器从零开始；点击“重载页面”，计数器重置为 0（因页面重载导致 `app.storage.client` 数据清空）。

四、典型应用场景与完整示例

场景 1：存储客户端专属临时缓存（减轻服务器压力）

需求：客户端加载页面时请求并缓存数据，同一页面访问期间重复使用缓存（如接口请求结果），页面重载后重新请求（不保留旧缓存）。

```
from nicegui import app, ui
import asyncio

# 模拟耗时接口请求（如数据库查询、第三方 API 调用）
async def fetch_data():
    await asyncio.sleep(1) # 模拟耗时 1 秒
    return {'user_info': {'name': 'Test User', 'id': 123}, 'timestamp':
    asyncio.get_event_loop().time()}

@ui.page('/')
def index():
    # 使用 app.storage.client 存储客户端专属临时缓存
    data = app.storage.client.get('user_data', None)
    if data is None:
        data = fetch_data()
        app.storage.client['user_data'] = data
    ui.label(f'用户信息: {data["user_info"]["name"]}')
```

```

async def index():
    ui.label('客户端专属临时数据缓存示例')

    # 检查缓存是否存在，存在则直接使用，不存在则请求数据
    if 'cached_data' not in app.storage.client:
        ui.notify('缓存不存在，正在请求数据...')
        data = await fetch_data()
        app.storage.client['cached_data'] = data # 存入客户端临时缓存
    else:
        data = app.storage.client['cached_data']
        ui.notify('使用客户端临时缓存数据')

    # 展示数据
    ui.label(f'用户信息: {data["user_info"]}')
    ui.label(f'数据获取时间戳: {data["timestamp"]:.2f}')

    # 清除缓存按钮（手动刷新数据）
    def clear_cache():
        if 'cached_data' in app.storage.client:
            del app.storage.client['cached_data']
            ui.notify('缓存已清除，刷新页面重新请求数据')
    ui.button('清除缓存', on_click=clear_cache)

    # 重载页面按钮（验证缓存是否重置）
    ui.button('重载页面', on_click=ui.navigate.reload)

ui.run()

```

优势：同一页面访问期间重复打开无需重新请求数据，减轻服务器压力；页面重载后缓存清空，确保获取最新数据。

场景 2：存储客户端专属资源密集型对象（如数据库连接）

需求：每个客户端连接时创建独立的数据库连接，用于页面交互中的数据操作，页面关闭 / 重载时自动释放连接，避免连接泄露。

```

from nicegui import app, ui
import sqlite3 # 示例：SQLite 数据库

@ui.page('/')
def index():
    ui.label('客户端专属数据库连接存储示例')

    # 初始化数据库连接（当前客户端专属，其他客户端不可见）
    if 'db_conn' not in app.storage.client:
        conn = sqlite3.connect('demo.db') # 创建数据库连接
        app.storage.client['db_conn'] = conn
        # 初始化测试表（若不存在）
        cursor = conn.cursor()
        cursor.execute('CREATE TABLE IF NOT EXISTS users (id INT, name TEXT)')
        conn.commit()
    ui.notify('客户端专属数据库连接已创建')

```

```

# 插入测试数据
def insert_data():
    conn = app.storage.client['db_conn']
    cursor = conn.cursor()
    cursor.execute('INSERT INTO users (id, name) VALUES (?, ?)', (1, 'New User'))
    conn.commit()
    ui.notify('数据插入成功')

# 查询数据
def query_data():
    conn = app.storage.client['db_conn']
    cursor = conn.cursor()
    cursor.execute('SELECT * FROM users')
    results = cursor.fetchall()
    ui.notify(f'查询结果: {results}')

# 手动关闭连接（可选，页面结束后会自动释放）
def close_conn():
    if 'db_conn' in app.storage.client:
        app.storage.client['db_conn'].close()
        del app.storage.client['db_conn']
    ui.notify('数据库连接已关闭')

ui.button('插入数据', on_click=insert_data)
ui.button('查询数据', on_click=query_data)
ui.button('关闭连接', on_click=close_conn)
ui.button('重载页面', on_click=ui.navigate.reload)

ui.run()

```

核心价值：每个客户端独立连接，避免多客户端共享连接导致的并发冲突；页面重载 / 关闭时连接自动销毁，无需手动管理连接生命周期。

场景 3：存储页面级临时状态（如表单草稿，重载后重置）

需求：表单填写过程中临时保存草稿，同一页面访问期间误操作后可恢复，但页面重载后草稿清空（避免保留旧数据）。

```

from nicegui import app, ui

@ui.page('/')
def index():
    # 初始化表单草稿（从客户端临时存储读取，无则为空）
    default_form = app.storage.client.get('form_draft', {'name': '', 'email': ''})

    # 创建表单组件并绑定草稿数据
    name_input = ui.input('姓名', value=default_form['name'])
    email_input = ui.input('邮箱', value=default_form['email'])

    # 自动保存草稿（输入时更新存储）
    def save_draft():
        app.storage.client['form_draft'] = {

```

```
        'name': name_input.value,
        'email': email_input.value
    }
    ui.notify('草稿已保存')

# 绑定输入事件，实时保存草稿
name_input.on('input', save_draft)
email_input.on('input', save_draft)

# 恢复默认值（清空草稿）
def reset_form():
    app.storage.client['form_draft'] = {'name': '', 'email': ''}
    name_input.value = ''
    email_input.value = ''
    ui.notify('表单已重置')

# 提交表单
def submit_form():
    ui.notify(f'表单提交: 姓名={name_input.value}, 邮箱={email_input.value}')
    reset_form()

ui.button('提交表单', on_click=submit_form)
ui.button('重置表单', on_click=reset_form)
ui.button('重载页面', on_click=ui.navigate.reload)

ui.run()
```

效果：输入过程中实时保存草稿，刷新页面后草稿清空，确保每次重载都从空白表单开始。

五、与其他存储类型的核心区别

为明确 `app.storage.client` 的适用边界，以下对比其与 NiceGUI 其他四种存储类型的关键差异：

对比维度	.client	.tab	.user	.browser	.general
存储位置	服务器内存	服务器内存	服务器（文件 / Redis）	浏览器 Cookie	服务器（文件 / Redis）
跨标签页	否（独立）	否（独立）	是（共享）	是（共享）	是（共享）
跨页面重载	否（清空）	是（保留）	是（保留）	是（保留）	是（保留）
跨服务器重启	否	是（标签页未关）	是	否	是
数据类型支持	任意 Python 对象	任意 Python 对象	序列化类型	序列化类型	序列化类型
需客户端连接	否	是	否	否	否
需 storage_secret	否	否	是	是	否

对比维度	.client	.tab	.user	.browser	.general
核心生命周期	页面访问周期	标签页会话周期	用户会话周期	浏览器会话周期	应用生命周期

关键选型建议

- 需客户端临时存储、页面重载后清空 → 选 `app.storage.client`；
- 需标签页独立存储、页面重载后保留 → 选 `app.storage.tab`；
- 需用户级跨标签页共享、长期保留 → 选 `app.storage.user`；
- 需浏览器端存储、无服务器依赖 → 选 `app.storage.browser`（不推荐，优先 `user`）；
- 需全局所有用户共享、应用级配置 → 选 `app.storage.general`。

六、最佳实践与注意事项

1. 最佳实践

- **存储短期临时数据**：优先用于存储仅需在当前页面访问期间使用的数据（如缓存、临时状态、资源对象），避免存储需长期保留的数据；
- **资源对象优先用**：充分利用其“支持任意 Python 对象”的优势，存储数据库连接、流对象、大型缓存结果等，避免浏览器存储的限制；
- **无需手动清理**：依赖框架的自动生命周期管理，无需编写额外的清理代码（如页面卸载时删除数据），简化开发；
- **避免存储敏感数据**：数据存储在服务器内存，但页面访问结束后自动销毁，若需存储敏感数据（如用户令牌），建议用 `app.storage.user` 并配合 `storage_secret` 加密。

2. 注意事项

- **页面重载即清空**：核心特性也是易踩坑点 —— 页面重载、导航到其他页面都会导致数据丢失，需确认业务场景是否允许该行为；
- **内存占用控制**：若大量客户端同时连接且存储大型对象，可能导致服务器内存占用过高，需合理控制存储数据的大小和数量；
- **无持久化保障**：服务器崩溃、客户端网络中断时，数据会直接丢失，不适合存储关键业务数据；
- **客户端隔离边界**：同一浏览器的不同标签页视为不同客户端，数据完全隔离，若需跨标签页共享，需改用 `app.storage.user` 或 `app.storage.browser`。

七、总结

`app.storage.client` 是 NiceGUI 针对“客户端短期临时存储”场景的最优解，核心优势在于**客户端专属隔离、支持任意对象存储、自动生命周期管理、无依赖快速上手**。其适用场景集中在：

1. 客户端临时缓存（如接口请求结果、页面加载数据）；
2. 资源密集型对象存储（如数据库连接、流对象）；
3. 页面级临时状态（如表单草稿、交互中间结果）；
4. 需页面重载后恢复默认状态的场景。

与其他存储类型相比，`app.storage.client` 填补了“服务器端、客户端专属、短期临时”的存储空白，既避免了浏览器存储的序列化限制和容量约束，又无需手动管理数据生命周期，是提升应用性能、简化临时数据处理的核心工具。使用关键在于明确其“页面访问周期”的生命周期边界，避免用于需长期保留或跨标签页共享的数据场景。